
Modelos de regresión avanzados

— M. en C. Gabriela Martínez —

Unidad I

Objetivo de la unidad

1. Comprender el pipeline completo de modelado avanzado.
2. Representar datos y modelos con vectores, matrices y funciones de pérdida.
3. Explicar la regresión lineal como modelo base para estimar una respuesta continua.
4. Extender un modelo lineal mediante transformaciones no lineales.
5. Identificar riesgos de fuga de datos, mala validación, multicolinealidad y sobreajuste.

Pipeline de modelado avanzado

- Modelado avanzado: del problema a la validación.
- Repaso matemático aplicado: vectores, matrices y pérdida.
- Regresión lineal: modelo, estimación e interpretación.
- Sustitución no lineal: transformar variables para capturar curvatura.

Hilo conductor: formular una pregunta, representar datos, ajustar un modelo, validar resultados y decidir si el modelo generaliza.

Idea clave: antes de hablar de algoritmos, debe quedar claro qué se predice, con qué datos y cómo se valida.

Pipeline de modelado avanzado

Flujo mínimo

1. Plantear problema y decisión.
2. Definir variable objetivo.
3. Integrar y limpiar datos.
4. Explorar patrones y calidad.
5. Diseñar variables predictoras.
6. Separar train, validación y test.
7. Entrenar, comparar y comunicar.

Errores frecuentes

- Empezar por el algoritmo sin entender el contexto del problema
- Usar variables que no existirían al momento de predecir
- Medir desempeño con los mismos datos de entrenamiento
- Elegir métricas que no reflejan el costo real del error
- Comunicar métricas sin supuestos o límites

Del problema a la métrica

Una pregunta analítica debe convertirse en una salida observable.

Ejemplo:

- ¿cuánto consumo tendrá un usuario la próxima semana?
- ¿un cliente abandonará?
- ¿qué perfiles de usuarios existen?

Baseline: solución simple que sirve como referencia. En regresión puede ser predecir la media; en clasificación, la clase mayoritaria.

Discusión: una métrica incorrecta puede hacer que un modelo parezca bueno aunque sea inútil

Métricas según el contexto

La métrica debe reflejar el tipo de problema, la decisión que se tomará y el costo real del error.

Caso	Métrica útil	Por qué sirve	Métrica inútil
Predecir consumo Regresión	MAE si importa el error promedio. RMSE si errores grandes son más costosos. MAPE si conviene comparar porcentajes.	Evalúa qué tan lejos queda la predicción del valor real en la misma escala del negocio.	Accuracy: no hay clases correctas o incorrectas. R2 aislado: puede ocultar errores grandes relevantes.
Predecir abandono Clasificación	Recall si perder usuarios en riesgo es caro. Precisión si contactar falsos positivos cuesta. F1 o PR-AUC si hay desbalance.	Se enfoca en el tipo de error que afecta la decisión: no detectar riesgo o intervenir de más.	Accuracy con 95% de no abandono: puede parecer alta sin detectar a quienes sí abandonan.
Perfilar usuarios Agrupamiento	Silueta para separación/cohesión. Estabilidad de segmentos. Utilidad interpretativa para decidir acciones.	No basta separar puntos: los grupos deben ser consistentes, explicables y accionables.	Accuracy o R2: no existen etiquetas verdaderas ni respuesta continua que comparar.

Validación y fuga de datos

1. Train: ajusta parámetros del modelo.
2. Validación: compara modelos e hiperparámetros.
3. Test: estima desempeño final con datos no usados para decidir.

Fuga de datos: ocurre cuando el entrenamiento usa información que no estaría disponible al momento real de predicción.

- Ejemplo: predecir abandono usando una variable registrada después de que el usuario ya abandonó.

Regla práctica: validar debe simular lo mejor posible el uso futuro del modelo.

Fundamento matemático base

- Observación: una fila de datos que describe un caso.
- Característica: una variable de entrada usada por el modelo.
- Matriz X : contiene n observaciones y p características.
- Vector y : contiene la respuesta observada.
- Vector β : contiene los parámetros que el modelo debe estimar.

La meta del modelado es elegir parámetros que produzcan buenas predicciones en datos no vistos.

Análisis dimensional base

$$X \in R^{n \times p}, \quad y \in R^{n \times 1}$$

$$\hat{y} = X\beta \Rightarrow (n \times p)(p \times 1) = (n \times 1)$$

- **X** contiene n registros y p características originales.
- **y** contiene una respuesta por cada registro.
- Para que **Xβ** produzca n predicciones, **β** debe tener p filas y 1 columna.
- El producto matricial define la compatibilidad de dimensiones.

Modelo lineal: un parámetro por característica

$$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

$$X^* \in R^{n \times (p+1)}, \quad \beta \in R^{(p+1) \times 1}$$

- Si se incluye intercepto, se agrega una columna de unos a X .
- El vector de parámetros contiene β_0 más p coeficientes.
- La dimensión de β depende de p , no de n .

Modelo polinomial: depende del grado

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_d x^d$$

$$\hat{y} = \Phi(X)\beta, \quad \Phi(X) \in R^{n \times m}$$

- Con una variable y grado d , hay d términos más el intercepto.
- Con varias variables, $\Phi(X)$ representa potencias, interacciones y sustituciones.
- Si $\Phi(X)$ tiene m columnas finales, β tiene m parámetros; o $m+1$ si el intercepto se maneja aparte.

Red neuronal: pesos y sesgos

$$W_1 \in R^{p \times h}, b_1 \in R^{h \times 1}, W_2 \in R^{h \times 1}, b_2 \in R^{1 \times 1}$$

$$\#\theta = ph + h + h + 1 = ph + 2h + 1$$

- En una red neuronal no hay un único β : usamos θ para reunir todos los pesos W y sesgos b .
- Ejemplo: p entradas, h neuronas ocultas y una salida continua.
- La arquitectura fija cuántas conexiones y sesgos se estiman.

Actividad

Se desea predecir los minutos de reproducción semanal de usuarios de una plataforma de streaming.

La matriz X tiene 120 registros y 4 características:

antigüedad, sesiones por semana, minutos previos y calificación promedio.

Se propone una red neuronal totalmente conectada con una capa oculta de 3 neuronas y una salida continua.

Ejercicio: determina las dimensiones de W_1 , b_1 , W_2 y b_2 , y calcula el número total de parámetros.

Revisión de actividad: arquitectura propuesta

¿La arquitectura propuesta es adecuada para la estimación de ese conjunto de datos?

Regla para calcular dimensiones

1. Identificar el aproximador: lineal, polinomial, red neuronal u otro.
2. Determinar qué entradas reales recibe el modelo.
3. Contar términos, columnas transformadas, conexiones y sesgos.
4. Ver si el intercepto o sesgo se incluye como parámetro adicional.
5. Escribir la dimensión para que la operación produzca una salida $n \times 1$.

Idea clave: decir el tipo de aproximador permite calcular dimensiones porque define la estructura de parámetros.

Repaso interactivo

Fundamentos de Regresión y Complejidad del Modelo

Regresión lineal simple

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$$

Usa una sola variable explicativa para aproximar una respuesta continua.

β_0 representa el intercepto; β_1 representa la pendiente.

ε_i concentra el error que el modelo no explica.

OLS (mínimos cuadrados ordinarios)

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$
$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

OLS elige la recta que minimiza la suma de errores cuadrados.

La pendiente compara covariación entre x e y contra variación de x.

El intercepto ubica la recta en el nivel promedio de y.

Notación matricial - Regresión lineal

$$\hat{y} = X\beta$$

$$e = y - \hat{y}$$

X - transforma cada observación en una combinación de características.

β - asigna peso a cada característica.

$\hat{y}$ - es la predicción producida por el modelo.

e - es el residuo: lo que el modelo no explicó.

Cuando agregamos una columna de unos a X , β incluye el intercepto.

Ejercicio: de OLS a la pérdida

Caso: predecir minutos de reproducción semanal.

Variables disponibles antes de predecir:
sesiones por semana, minutos vistos la semana anterior y antigüedad del usuario.

Modelo: $y = X\beta + \varepsilon$

Pregunta guía:

¿Qué valores de β hacen que $\hat{y} = X\beta$ quede lo más cerca posible de y ?

OLS responde minimizando la suma de errores cuadrados.

$$\text{Residuo: } e = y - \hat{y} \quad | \quad \text{Pérdida: } L(\beta) = \sum (y_i - \hat{y}_i)^2$$

Regresión lineal múltiple

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \varepsilon_i$$

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \cdots + \hat{\beta}_p x_{ip}$$

Usa p variables explicativas al mismo tiempo.

Cada β_j estima el cambio esperado en y cuando x_j cambia una unidad, manteniendo las demás variables constantes.

La interpretación depende de escala, correlación entre variables y supuestos del modelo.

OLS en forma matricial

$$y = X\beta + \varepsilon$$

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

$$\hat{y} = X\hat{\beta}, \quad e = y - \hat{y}$$

La solución cerrada existe si $X^T X$ es invertible.

En la práctica, el software usa métodos numéricos más estables.

El objetivo sigue siendo minimizar la pérdida cuadrática.

Regresión con sustitución no lineal

Una sustitución no lineal transforma las variables originales para representar relaciones curvas o interacciones.

La clave: el modelo puede ser no lineal respecto a x , pero seguir siendo lineal respecto a β .

Ejemplo:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + e$$

Aunque aparece x^2 , los parámetros β entran de forma lineal.

Transformaciones, polinomios e interacciones

Opciones comunes

Polinomios: x , x^2 , x^3 .

Logaritmos: $\log(x)$, útiles para crecimiento proporcional.

Raíces: reducen efecto de valores grandes.

Interacciones: x_1x_2 , cuando el efecto de una variable depende de otra.

Funciones base: transformaciones diseñadas para capturar patrones específicos.

Interpretación

- La variable transformada se vuelve una nueva característica.
- El coeficiente ya no se interpreta como un cambio simple y constante en la escala original.
- Conviene visualizar predicciones y residuales para explicar el patrón aprendido.
- La transformación debe tener sentido para el fenómeno, no solo mejorar la métrica.

R^2 : coeficiente de determinación

R^2 mide qué proporción de la variabilidad de y logra explicar el modelo.

Interpretación:

- $R^2 = 0$: no mejora frente a predecir la media.
- $R^2 = 0.75$: explica cerca del 75% de la variabilidad observada.
- $R^2 = 1$: ajuste perfecto en los datos usados.

No acertamos exactamente al 75% de los usuarios; reducimos el error cuadrático un 75% respecto a la referencia.

$$R^2 = 1 - \frac{SSE}{SST}$$

$$SSE = \sum (y_i - \hat{y}_i)^2$$

$$SST = \sum (y_i - \bar{y})^2$$

y_i : valor observado

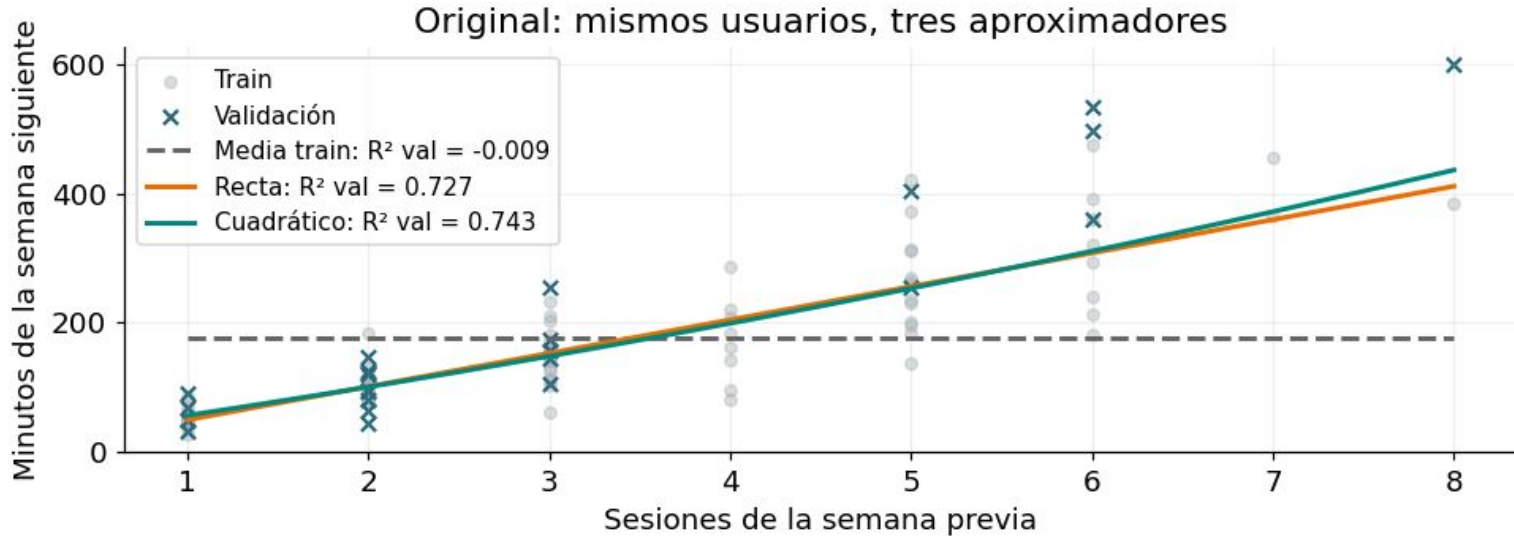
$\hat{y}_i$: predicción del modelo

$\bar{y}$: media observada

Sumas sobre los n registros

R^2 : media, recta y polinomio

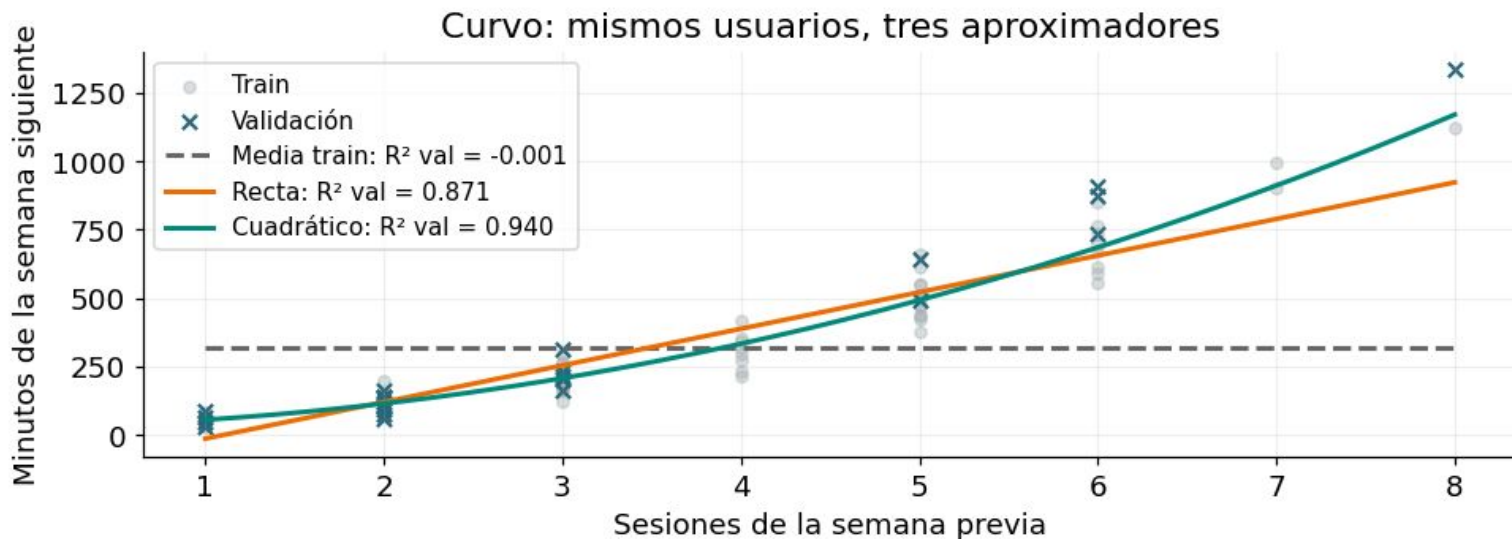
Respuesta original; 72 usuarios de entrenamiento y 24 de validación.



Recta: RMSE = 84.62 min. Cuadrático: 82.02 min. La mejora es pequeña.

Cuando la respuesta tiene curvatura

Escenario controlado: $y_{\text{curvo}} = y_{\text{original}} + 15(x - 1)^2$.



$R^2 \text{ val}: 0.871 \rightarrow 0.940$. RMSE: 120.80 $\rightarrow$ 82.02 min, de recta a cuadrático.

Recta vs. curva: un ejemplo con datos

1D lineal: $20 + 50x$

1D cuadrática: $20 + 50x + 15(x - 1)^2$

Por sesión adicional:

- Recta +50
- Curva +65, +95, +125, +155 mins

Sesiones (x)	Recta (min)	$15(x - 1)^2$	Curva (min)
1	70	0	70
2	120	15	135
3	170	60	230
4	220	135	355
5	270	240	510

Lineal vs. cuadrático: ¿qué dice R^2 ?

Misma respuesta curva y mismos 24 usuarios de validación.

x = sesiones previas; x^2 no es otra característica original.

Modelo	Polinomio en sesiones	R^2 val	RMSE val
Lineal (grado 1)	$\hat{y} = \beta_0 + \beta_1 x$	0.871	120.80 min
Cuadrático 1D	$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$	0.940	82.02 min

$R^2 \approx 0.94$: $SSE \approx 0.06 \times SST$ en validación.

94% menos error cuadrático que la referencia de su media.

No es 94% de aciertos; $RMSE \approx 82$ min.

¿Faltan variables o falta curvatura?

Una característica: sesiones. Su cuadrado cambia la forma, no añade información de otra variable.

Regresión múltiple: incorpora varias características originales; no agrega curvatura automáticamente.

El polinomio 1D ya logra $R^2 \approx 0.94$ en el escenario curvo.

No ajustarlo todo no demuestra fracaso: hay ruido y necesitamos una tolerancia de error en minutos.

Comprobar con varias características

Mismo objetivo curvo. z_1 : antigüedad; z_2 : sesiones; z_3 : historial; z_4 : calificación.
Entradas estandarizadas; q incluye el intercepto. Cada fila estima su propio β .

Modelo	Expresión de $\hat{y}$	q	RMSE (min)	R^2 val
Simple	$\beta_0 + \beta_1 z_2$	2	120.80	0.871
Polinomial 1D	$\beta_0 + \beta_1 z_2 + \beta_2 z_2^2$	3	82.02	0.940
Múltiple (4)	$\beta_0 + \beta_1 z_1 + \beta_2 z_2 + \beta_3 z_3 + \beta_4 z_4$	5	106.52	0.899
Múltiple + z_2^2	$\beta_0 + \beta_1 z_1 + \beta_2 z_2 + \beta_3 z_3 + \beta_4 z_4 + \beta_5 z_2^2$	6	79.26	0.944

Más características ayudan poco aquí; la **curvatura** aporta la mayor mejora.

Interpretación y diagnóstico del modelo

- **Coefficientes:** indican dirección y magnitud del efecto estimado
- **Residuales:** muestran errores y patrones que el modelo no capturó
- **R^2 :** proporción de variabilidad explicada; no prueba causalidad
- **Multicolinealidad:** variables predictoras muy relacionadas pueden volver inestables los coeficientes
- **Perturbaciones:** algunos puntos pueden dominar el ajuste

Pregunta para análisis: ¿un R^2 alto garantiza que el modelo servirá con datos nuevos?

Complejidad, sobreajuste y validación

Aumentar el grado polinomial aumenta flexibilidad, pero también riesgo de ajustar ruido.

- Grado 1: patrón lineal, simple, interpretable, posible subajuste.
- Grado 2 o 3: captura curvatura moderada si existe evidencia.
- Grado alto: puede seguir variaciones locales del entrenamiento y fallar con datos nuevos.

Decisión: elegir complejidad con validación, residuales y sentido del problema.

Regresión robusta en varias dimensiones

Entradas: antigüedad, sesiones, historial y calificación.

Añadimos sesiones² para conservar la curvatura del problema.

$$\hat{y} = \beta_0 + \sum_{j=1}^4 \beta_j z_j + \beta_5 z_2^2$$

$$A = [\mathbf{1}, z_1, z_2, z_3, z_4, z_2^2], \quad \beta \in \mathbb{R}^{6 \times 1}$$

Diferentes funciones de pérdida pueden usar las mismas 6 características y el mismo objetivo.

Regresión robusta

Ya elegimos variables y una forma de aproximación. Ahora revisamos la **sensibilidad** del ajuste.

Unos pocos errores grandes pueden dominar la pérdida cuadrática.

La regresión robusta reduce ese efecto **cambiando la penalización**.

No reemplaza la búsqueda de variables útiles ni corrige automáticamente el leverage.

Atípicos, leverage e influencia

Atípico en respuesta: se aleja de lo esperado en y dadas sus características.

Leverage: tiene una posición en X inusual en el espacio de todas las entradas.

Influencia: al retirarlo cambian los coeficientes o las predicciones.

No son categorías excluyentes.

- Revisamos residuos, leverage, Cook y cambio de predicción.

Perturbación de datos en varias dimensiones

Tres registros añadidos a train.

- Comparten 28 meses de antigüedad, 3 sesiones previas y calificación 2.6.

Caso	Historial	Predicción base	Observado (y)	Diferencia
A	27 214	209.24	1409.24	+1200
L	1 000 000	927.85	927.85	0
I	1 000 000	927.85	2127.85	+1200

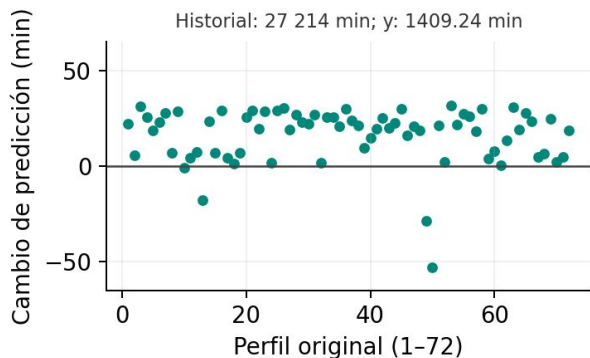
Valores en minutos. Diferencia = observado – predicción base.

Historial máximo original: 124 875 min.

Leverage no implica influencia

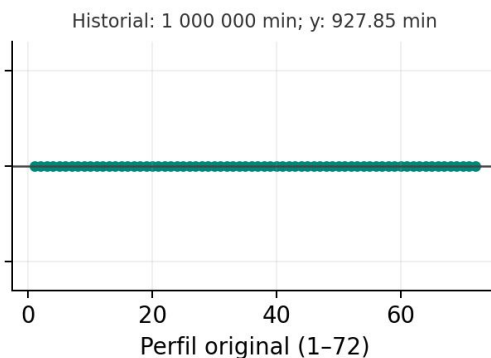
Cada panel: 72 registros originales + 1 extra. La estrella identifica el añadido.
Los puntos de la gráfica son los cambios en los 72 perfiles originales.

★ A añadido



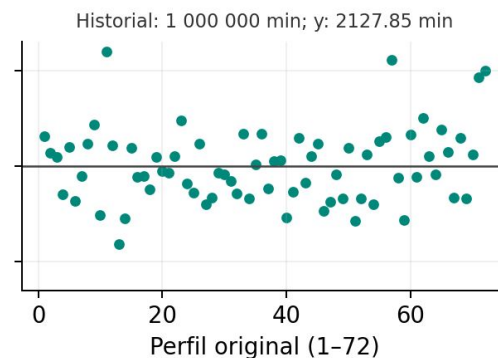
Influencia visible

★ L añadido



Casi no cambia el ajuste

★ I añadido



Influencia muy grande

Optimizar: misma predicción, otra pérdida

Elegimos β para minimizar la suma de penalizaciones de los residuos.

$$\hat{\beta}_{OLS} = \arg \min_{\beta} \sum_i \frac{1}{2} (y_i - \hat{y}_i)^2$$

$$\hat{\beta}_{Huber} = \arg \min_{\beta} \sum_i L_{\delta} (y_i - \hat{y}_i)$$

OLS: Todos los errores crecen al cuadrado.

Huber: Los errores grandes pasan a crecimiento lineal.

Huber: cuadrática cerca, lineal lejos

$r = y - \hat{y}$, en minutos. δ marca dónde cambia la penalización.

Usaremos $\delta = 30$ min solo en el ejemplo aritmético.

$$L_{\delta}(r) = \frac{1}{2}r^2 \quad \text{si } |r| \leq \delta$$

$$L_{\delta}(r) = \delta(|r| - \frac{\delta}{2}) \quad \text{si } |r| > \delta$$

Si $|r| \leq \delta$: como OLS. Si $|r| > \delta$: cada minuto adicional cuesta lo mismo.

El valor extremo sigue participando, pero recibe menos peso relativo.

Por qué un error extremo pesa menos

La función cambia después de ± 30 min; conserva continuidad.



Huber no acota la pérdida: acota su pendiente respecto al residuo.

Ejemplo: penalizar el mismo error

Con $\delta = 30$ min, comparamos pérdidas antes de reajustar.

Residuo (min)	OLS: $r^2/2$	Huber: $\delta = 30$
10	50	50
100	5000	2550
1200	720 000	35 550

Para $r = 100$: $30(100 - 15) = 2550$.

Un error de 1200 min sigue penalizado, pero deja de dominar al cuadrado.

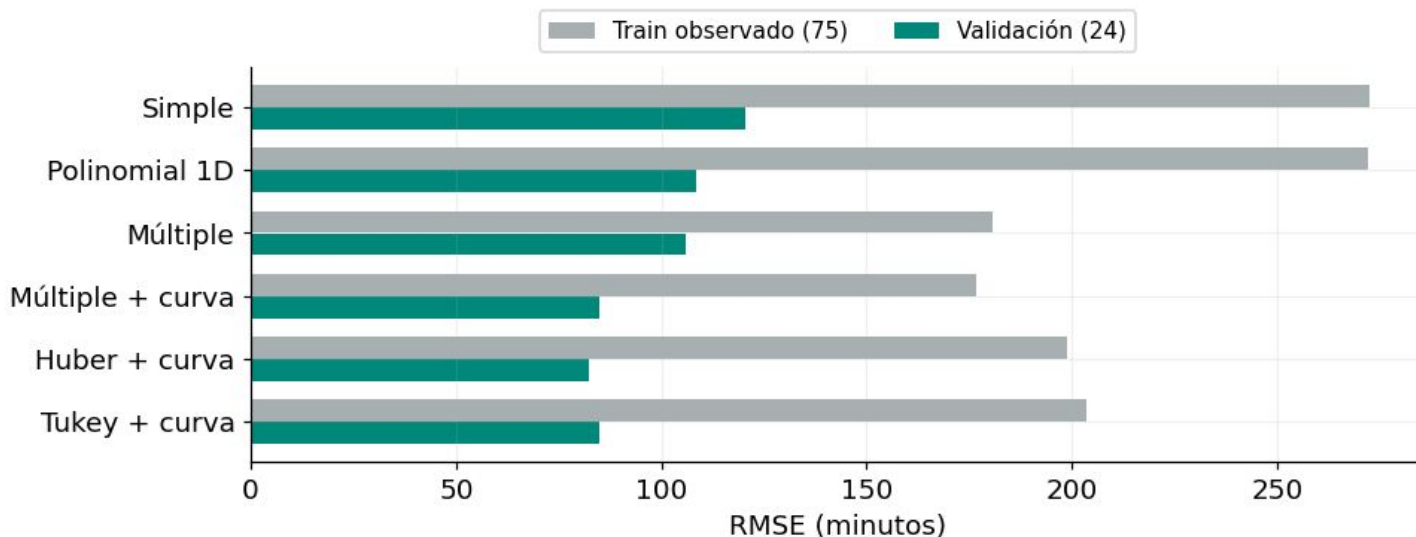
Del ejemplo al ajuste de Huber

1. Ajustar un modelo inicial y calcular sus residuos.
2. Estimar s : tamaño habitual del error, resistente a extremos.
3. Usar $\delta = 1.345s$; calcular pesos y volver a ajustar.
4. Repetir hasta estabilizar el ajuste.

En esta ejecución: $s \approx 40.09$ min y $\delta \approx 53.92$ min.

Entrenamiento y validación: mismos métodos

Todos usan 75 registros observados de train y los mismos 24 de validación.



La comparación OLS múltiple + curva / Huber aísla el cambio de pérdida.

Cambiar la pérdida: ¿qué mejora?

Mismas 6 columnas; solo cambia cómo elegimos los coeficientes.

Comparación de RMSE en minutos.

Método	Train observado	Validación
OLS	176.72	84.91
Huber	199.05	82.26
Tukey	203.56	84.75

Huber acepta más error en train y mejora **modestamente** en validación.

No prueba superioridad universal; el test permanece reservado.

Del diagnóstico a los pesos robustos

No eliminamos los registros: el ajuste cambia su contribución.

Punto	Peso Huber	Peso Tukey
A	0.045	0.000
L	1.000	1.000
I	0.047	0.000

Peso bajo no confirma un error de registro.

Investigar el contexto sigue siendo parte del análisis.

Ampliación: Huber y Tukey

Una familia de estimadores definida por la pérdida ρ .

Huber crece linealmente; **Tukey** es acotada y puede dar peso cero.

$$\hat{\beta} = \arg \min_{\beta} \sum_{i=1}^n \rho(u_i), \quad u_i = \frac{y_i - \phi(x_i)^T \beta}{s}$$

$$\psi(u) = \rho'(u), \quad w_i = \frac{\psi(u_i)}{u_i}$$

IRLS: residuos $\rightarrow$ escala $\rightarrow$ pesos $\rightarrow$ reajuste. Para $u = 0$, usamos $w = 1$.

Tukey ($cT = 4.685$) no es convexa: importa la inicialización.

Aproximadores universales

Teorema: una red con una capa oculta suficientemente ancha, sesgos y una activación continua no polinómica puede aproximar una función continua en un conjunto compacto.

$$\sup_{x \in K} |f(x) - \hat{f}(x)| < \varepsilon$$

Para cada función f y tolerancia $\varepsilon > 0$, existe una red adecuada.

ReLU, tanh y sigmoide son ejemplos; no se exige discontinuidad.

Aproximar, aprender y generalizar

Capacidad: existe una función de la familia que aproxima el fenómeno.

Optimización: el entrenamiento debe encontrar buenos parámetros.

Generalización: el modelo debe funcionar con datos no vistos.

Una red más grande no garantiza resolver las tres tareas.

De bases diseñadas a bases aprendidas

Regresión lineal: usamos las características originales.

Polinomial: diseñamos potencias e interacciones.

Red neuronal: aprende transformaciones mediante pesos y activaciones.

La pérdida sigue siendo otra decisión: cuadrática, Huber u otra.

Red neuronal para regresión

Una **capa oculta** transforma las entradas; la salida combina sus activaciones.

Sin **activación no lineal**, varias capas equivalen a un modelo afín.

$$H = \sigma(XW_1 + \mathbf{1}b_1), \quad \hat{y} = HW_2 + \mathbf{1}b_2$$

$$X : n \times p \quad W_1 : p \times h \quad b_1 : 1 \times h \quad H : n \times h$$

$W_2 : h \times 1$; $b_2 : 1 \times 1$. Parámetros: $p \times h + h + h + 1$.

Con $p = 4$ y $h = 3$: 19 parámetros; salida de un valor por usuario.

Cómo aprende una red

Propagación hacia delante: calcular predicciones y pérdida.

Backpropagation: aplicar la regla de la cadena para calcular gradientes.

$$L(\theta) = \frac{1}{n} \sum_i \rho(y_i - \hat{y}_i)$$

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

El optimizador actualiza θ (pesos y sesgos); η controla el paso.

Vigilar pérdida de entrenamiento y validación en cada época.

Redes en ciencia de datos

Preparar: imputar y escalar con entrenamiento; evitar fuga temporal.

Comparar: usar las mismas particiones y métricas que OLS y polinomios.

Controlar: regularización, parada temprana y varias semillas.

Explicar: analizar residuos, sensibilidad y resultados por tipo de usuario.

Redes avanzadas: elegir por estructura

MLP profundo: compone capas para aprender interacciones en datos tabulares.

RNN/LSTM o atención: permiten modelar secuencias cuando conservamos el orden de las sesiones.

Conexiones residuales: facilitan optimizar redes profundas.

Más profundidad no sustituye más evidencia ni mejores variables.

Redes de funciones de base radial

Cada unidad responde según la **distancia** de x a un centro c_j .

Una base gaussiana activa una **región local** del espacio de entrada.

$$\phi_j(x) = \exp\left(-\frac{\|x - c_j\|^2}{2\sigma_j^2}\right), \quad \sigma_j > 0$$

$$\hat{y} = \beta_0 + \sum_{j=1}^m \beta_j \phi_j(x)$$

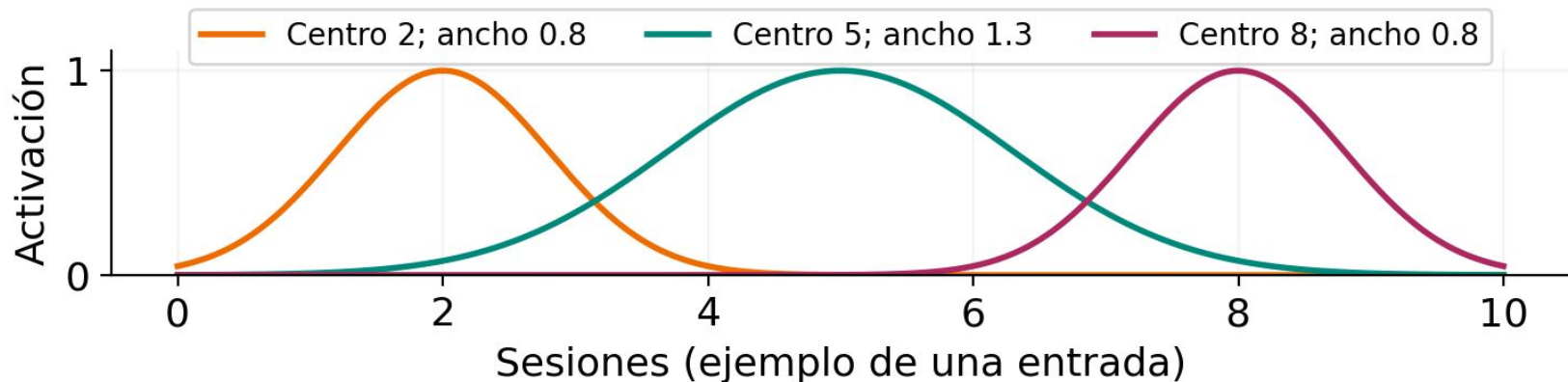
Centro: dónde responde la unidad. Ancho: qué tan lejos responde.

Los coeficientes β combinan esas respuestas para predecir minutos.

RBF: centros, anchos y cobertura

Anchos pequeños: respuestas muy locales; riesgo de ajustar ruido.

Anchos grandes: respuestas suaves; riesgo de perder detalles.



RBF: ajuste y número de parámetros

Fijar m centros y anchos: construir Φ con una columna por base.

Añadir intercepto: $A = [1, \Phi]$ tiene $n \times (m + 1)$ entradas.

Ajustar salida: estimar $m + 1$ coeficientes con OLS o una pérdida robusta.

Si también se entrenan centros y anchos: $m_p + 2m + 1$ parámetros.

Streaming: ¿qué aproximador conviene?

Polinomio: pocas bases diseñadas y un ajuste fácil de inspeccionar.

RBF: bases locales; revisar cobertura y escalas de las variables.

MLP: representaciones aprendidas; mayor cuidado al entrenar y validar.

Reto: ¿qué evidencia justificaría pasar del polinomio a una red?